Sistema WEB de Gestão de Oficina de Games

PixelFix Games

Documentação técnica: requisitos, infraestrutura, modelagem, banco de dados e plano de testes

1. Visão geral

A PixelFix Games é uma assistência técnica de consoles de videogame e notebooks gamer. O sistema web resolve os problemas apontados: trocas de aparelhos, conflitos de horário nas bancadas, clientes duplicados e perda do histórico de reparos, além de proteger os dados sensíveis (CPF, senha e número de série) conforme a LGPD.

O sistema é composto por uma API REST (Node.js + Express + Sequelize + MySQL) e um frontend em HTML/CSS/JavaScript puro, que consome a API com token JWT.

2. Requisitos

2.1 Requisitos Funcionais (RF)

ID	Nome	Descrição
RF01	Cadastro de cliente	Permitir cadastrar cliente com nome, e-mail, senha, CPF e telefone celular (rota pública de auto-cadastro).
RF02	Login	Autenticar o cliente por e-mail e senha, retornando token de sessão com expiração e mensagens de erro claras.
RF03	Logout	Encerrar a sessão do cliente a partir do menu superior, removendo o token e voltando ao login.
RF04	Página principal	Exibir a página principal identificando o cliente logado e o horário de expiração da sessão.
RF05	Menu de navegação	Apresentar menu no topo de todas as telas, com acesso a Clientes, Consoles e Agendamentos.
RF06	Listar clientes	Listar todos os clientes (CPF mascarado) com ações de editar e excluir.
RF07	Atualizar cliente	Consultar e atualizar os dados de um cliente; a senha só é alterada se uma nova for informada.
RF08	Excluir cliente	Excluir cliente, respeitando as regras de negócio de integridade (RN09 e RN10).
RF09	Cadastrar console	Cadastrar console (modelo, marca, nº de série, tipo de serviço) vinculado a um cliente existente.
RF10	Listar consoles	Listar todos os consoles com o nome do cliente, exibindo o nº de série mascarado.
RF11	Consultar console por termo	Pesquisar consoles por termo: número do console (código), nº de série, modelo ou marca.
RF12	Exibir nº de série completo	Permitir ao usuário autenticado visualizar o nº de série descriptografado de um console específico.

ID	Nome	Descrição
RF13	Cadastrar agendamento	Registrar agendamento de manutenção (data e hora) vinculado a cliente e console.
RF14	Atualizar agendamento	Alterar data/hora, cliente ou console de um agendamento existente.
RF15	Listar agendamentos	Listar agendamentos ordenados pela data/hora agendada (crescente).
RF16	Filtrar consoles por cliente	No agendamento, listar para seleção apenas os consoles do cliente escolhido.
RF17	Expiração de sessão	Ao expirar a sessão (90 min), redirecionar automaticamente ao login informando o motivo.
RF18	Proteção de páginas e rotas	Impedir acesso às telas e à API (exceto login e auto-cadastro) sem token válido.

2.2 Requisitos Não Funcionais (RNF)

ID	Nome	Descrição
RNF01	Segurança - criptografia	Senha com hash bcrypt (salt, 10 rounds); CPF e nº de série com AES-256-CBC; nenhum desses dados é gravado em texto puro.
RNF02	Segurança - sessão	Autenticação por JWT assinado (HS256) com expiração de 90 minutos, enviado no cabeçalho Authorization: Bearer.
RNF03	Privacidade (LGPD)	CPF e nº de série são mascarados nas listagens; a versão completa só é devolvida na consulta individual autenticada. A senha nunca é devolvida.
RNF04	Usabilidade e acessibilidade	Menu com <nav> rotulado, link "pular para o conteúdo", foco visível, rótulos em todos os campos, mensagens em regiões aria-live.
RNF05	Responsividade	Telas utilizáveis em desktop e celular (menu com quebra de linha, tabelas com rolagem horizontal).
RNF06	Desempenho	Operações de CRUD e listagens devem responder em até 2 segundos em condições normais de uso.
RNF07	Manutenibilidade	Código em camadas (models, controllers, middleware, utils), configuração externa por arquivo .env.
RNF08	Confiabilidade	API retorna JSON padronizado com código HTTP adequado (400, 401, 404, 409, 500) e tratamento global de erros.
RNF09	Compatibilidade	Frontend funcional nos navegadores atuais (Chrome, Edge, Firefox) sem dependências externas.

2.3 Regras de Negócio (RN)

ID	Regra
RN01	E-mail do cliente deve ser único (comparado em minúsculas).
RN02	CPF deve ser válido (dígitos verificadores) e único - evita cadastro duplicado de clientes.
RN03	Nº de série do console deve ser único (5 a 30 caracteres: letras, números e hífen) - evita troca de aparelhos.
RN04	Todo console deve estar vinculado a um cliente existente.
RN05	O console de um agendamento deve pertencer ao cliente do agendamento.
RN06	Agendamentos só podem ser feitos para data/hora futura (na atualização, só é exigido se a data/hora for alterada).
RN07	Atendimento de segunda a sábado, das 08:00 às 18:00 (último horário às 17:59).
RN08	Intervalo mínimo de 60 minutos entre agendamentos (uma bancada) - evita erros de horário.

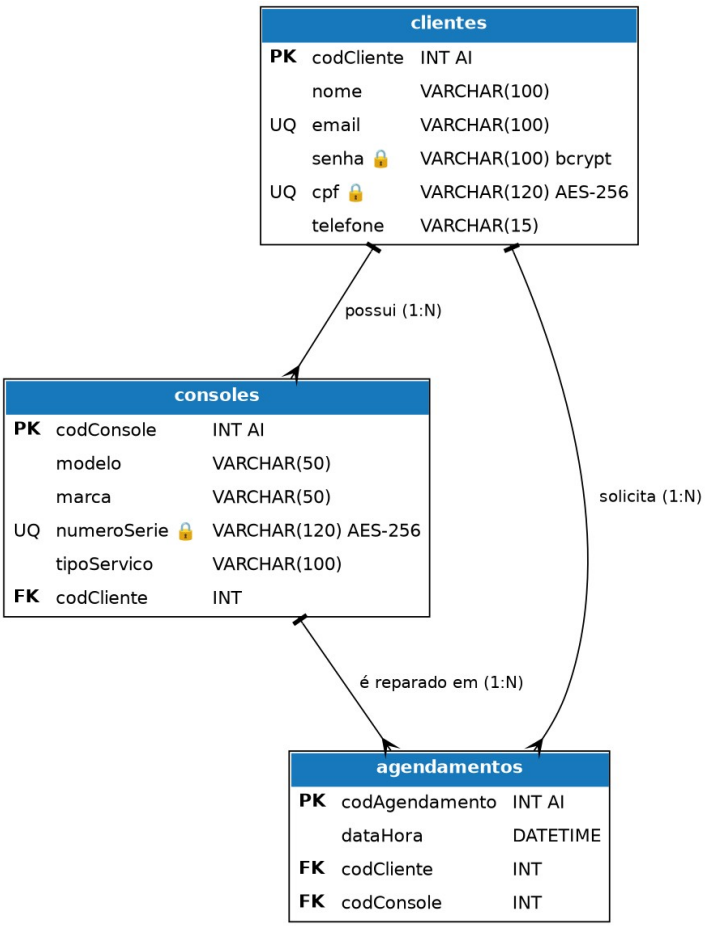
ID	Regra
RN09	Cliente com consoles vinculados não pode ser excluído (preservação do histórico de reparos).
RN10	O usuário logado não pode excluir a própria conta.
RN11	A sessão expira 90 minutos após o login.
RN12	Senha com no mínimo 6 caracteres.
RN13	Telefone deve ser celular brasileiro válido: DDD + 9 + 8 dígitos.

3. Requisitos de infraestrutura

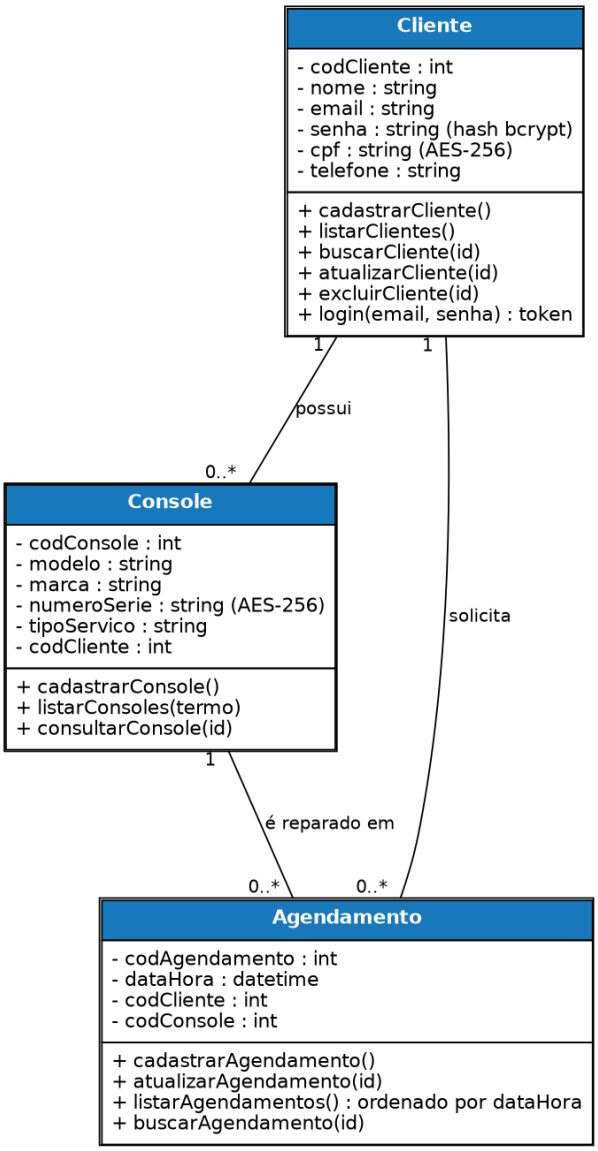
Item	Especificação
Banco de dados	MySQL 8.0 (nome do banco: db_pixelfix). O script também é compatível com MariaDB 10.6+; os testes desta entrega foram executados em MariaDB 10.11.
Linguagem (backend)	JavaScript em Node.js 22 LTS (testado na v22.22.2; mínimo recomendado: 20 LTS).
Framework / bibliotecas	Express 5.3, Sequelize 6.37 (ORM), mysql2 3.24, jsonwebtoken 9.0, bcryptjs 2.4, cors 2.8, dotenv 16.6.
Servidor web local (API)	Node.js + Express em http://localhost:3000 (npm start).
Servidor do frontend	Servidor estático local na porta 5500: "npx http-server frontend -p 5500" (ou extensão Live Server do VS Code). Acesso: http://localhost:5500/login.html.
Linguagens do frontend	HTML5, CSS3 e JavaScript (ES2020) sem frameworks; criptografia e regras ficam no servidor.
Teste de integração	REST Client (extensão do VS Code) com o arquivo backend/teste.http (equivalente em Postman/Insomnia) e suíte automatizada com node:test + fetch (npm test), 29 casos.
Cliente (navegador)	Chrome, Edge ou Firefox atuais.

4. Modelo lógico (DER)

Relacionamentos: um cliente possui vários consoles; um cliente solicita vários agendamentos; um console pode ter vários agendamentos. Os campos marcados com cadeado são gravados criptografados.


clientes
PK codCliente INT AI
nome VARCHAR(100)
UQ email VARCHAR(100)
senha 🔒 VARCHAR(100) bcrypt
UQ cpf 🔒 VARCHAR(120) AES-256
telefone VARCHAR(15)
possui (1:N)
consoles
PK codConsole INT AI
modelo VARCHAR(50)
marca VARCHAR(50)
UQ numeroSerie 🔒 VARCHAR(120) AES-256
tipoServico VARCHAR(100)
FK codCliente INT
solicita (1:N)
é reparado em (1:N)
agendamentos
PK codAgendamento INT AI
dataHora DATETIME
FK codCliente INT
FK codConsole INT

5. Diagrama de classes



6. Script do banco de dados

Arquivo completo (com dados de exemplo): backend/db/db_pixelfix.sql. Abaixo, a criação das tabelas.

```
DROP DATABASE IF EXISTS db_pixelfix;
CREATE DATABASE db_pixelfix CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE db_pixelfix;

CREATE TABLE clientes (
  codCliente INT NOT NULL AUTO_INCREMENT,
  nome       VARCHAR(100) NOT NULL,
  email      VARCHAR(100) NOT NULL,
  senha      VARCHAR(100) NOT NULL COMMENT 'hash bcrypt',
  cpf        VARCHAR(120) NOT NULL COMMENT 'AES-256-CBC (iv:hex)',
  telefone   VARCHAR(15)  NOT NULL,
  PRIMARY KEY (codCliente),
  UNIQUE KEY uq_clientes_email (email),
  UNIQUE KEY uq_clientes_cpf (cpf)
) ENGINE=InnoDB;

CREATE TABLE consoles (
  codConsole  INT NOT NULL AUTO_INCREMENT,
  modelo      VARCHAR(50)  NOT NULL,
  marca       VARCHAR(50)  NOT NULL,
```

```
  numeroSerie VARCHAR(120) NOT NULL COMMENT 'AES-256-CBC (iv:hex)',
  tipoServico VARCHAR(100) NOT NULL,
  codCliente  INT NOT NULL,
  PRIMARY KEY (codConsole),
  UNIQUE KEY uq_consoles_serie (numeroSerie),
  CONSTRAINT fk_consoles_cliente FOREIGN KEY (codCliente)
    REFERENCES clientes (codCliente) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE agendamentos (
  codAgendamento INT NOT NULL AUTO_INCREMENT,
  dataHora       DATETIME NOT NULL,
  codCliente     INT NOT NULL,
  codConsole     INT NOT NULL,
  PRIMARY KEY (codAgendamento),
  KEY idx_agendamentos_dataHora (dataHora),
  CONSTRAINT fk_agend_cliente FOREIGN KEY (codCliente)
    REFERENCES clientes (codCliente) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_agend_console FOREIGN KEY (codConsole)
    REFERENCES consoles (codConsole) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Dados de exemplo inseridos (os campos sensíveis estão criptografados no script):

Tabela	Registros
clientes (3)	Carlos Eduardo Silva (carlos@gmail.com) | Mariana Souza (mariana@gmail.com) | Rafael Lima (rafael@gmail.com). Senha de todos: 123456.
consoles (4)	PlayStation 5 / Sony (cliente 1) | Xbox Series X / Microsoft (cliente 2) | Switch OLED / Nintendo (cliente 3) | PlayStation 4 / Sony (cliente 1).
agendamentos (4)	10/11/2026 09:00 (cliente 1, console 1) | 10/11/2026 14:30 (cliente 2, console 2) | 11/11/2026 10:00 (cliente 3, console 3) | 12/11/2026 16:00 (cliente 1, console 4).

Observação: os valores cifrados do script foram gerados com a CRYPTO_KEY do arquivo .env.example. Se a chave for alterada, os dados de exemplo deixam de ser decifráveis; nesse caso, recadastre os dados pelo sistema.

7. Plano de teste de integração (API)

Objetivo: validar, de ponta a ponta (HTTP, regras de negócio e banco de dados), a autenticação, o CRUD de clientes, o cadastro e consulta de consoles e o agendamento, incluindo a criptografia dos dados sensíveis.

Ferramenta: REST Client (VS Code) com backend/teste.http para execução manual, e suíte automatizada backend/test/integracao.test.js (node:test), executada com "npm test". A suíte cria seus próprios dados e os remove ao final.

Pré-condições: banco db_pixelfix criado com o script SQL, arquivo .env configurado e dependências instaladas (npm install).

Resultado da execução: 29 testes executados, 29 aprovados, 0 falhas.

ID	Rota	Cenário	Esperado
T01	GET /	Verificar API no ar	200
T02	POST /cliente	CPF inválido (11111111111)	400
T03	POST /cliente	Cadastro válido; confere no banco que senha é hash bcrypt e CPF não está em texto puro	201
T04	POST /cliente	E-mail repetido e CPF repetido	409
T05	POST /login	Corpo vazio	400
T06	POST /login	Senha incorreta	401

ID	Rota	Cenário	Esperado
T07	POST /login	Credenciais válidas; token com exp - iat = 5400 s (90 min); sem campo senha na resposta	200
T08	GET /clientes	Sem token	401
T09	GET /clientes	Token expirado (campo expirado = true)	401
T10	GET /me	Retorna o cliente logado	200
T11	GET /clientes	Lista com CPF mascarado e sem senha	200
T12	GET /clientes/:id	CPF completo (descriptografado)	200
T13	PUT /clientes/:id	Atualiza nome e telefone	200
T14	GET /clientes/999999	Cliente inexistente	404
T15	POST /console	Cliente inexistente	404
T16	POST /console	Cadastro válido; nº de série cifrado no banco	201
T17	POST /console	Nº de série duplicado	409
T18	GET /consoles?termo=	Consulta por código do console e por nº de série; série mascarada	200
T19	GET /consoles/:id	Nº de série completo	200
T20	POST /agendamento	Data no passado	400
T21	POST /agendamento	Horário fora do expediente (22:00)	400
T22	POST /agendamento	Agendamento válido	201
T23	POST /agendamento	Conflito de horário (30 min após outro)	409
T24	POST /agendamento	Console pertencente a outro cliente	409
T25	PUT /agendamentos/:id	Altera horário	200
T26	GET /agendamentos	Lista ordenada por dataHora crescente	200
T27	DELETE /clientes/:id	Excluir o próprio usuário logado	409
T28	DELETE /clientes/1	Excluir cliente com console vinculado	409
T29	DELETE /clientes/:id	Excluir cliente sem vínculos	200

Rotas da API (todas as marcadas com "Sim" exigem Authorization: Bearer <token>):

Método	Rota	Auth	Descrição
GET	/	Não	Verifica se a API está no ar
POST	/login	Não	Autentica e devolve o token (90 min)
POST	/cliente	Não	Auto-cadastro de cliente
GET	/me	Sim	Cliente da sessão
GET	/clientes	Sim	Lista clientes (CPF mascarado)
GET	/clientes/:id	Sim	Consulta cliente
PUT	/clientes/:id	Sim	Atualiza cliente
DELETE	/clientes/:id	Sim	Exclui cliente
POST	/console	Sim	Cadastra console
GET	/consoles?termo=	Sim	Lista / consulta por termo
GET	/consoles/:id	Sim	Consulta console (série completa)
POST	/agendamento	Sim	Cadastra agendamento

Método	Rota	Auth	Descrição
GET	/agendamentos	Sim	Lista ordenada por data
GET	/agendamentos/:id	Sim	Consulta agendamento
PUT	/agendamentos/:id	Sim	Atualiza agendamento

8. Como executar

- Instale Node.js 20+ e MySQL 8; inicie o MySQL.
- Execute o script backend/db/db_pixelfix.sql (cria o banco, tabelas e dados de exemplo).
- Em backend/, copie .env.example para .env e ajuste usuário/senha do banco (padrão root/root).
- Em backend/: npm install e depois npm start (API em http://localhost:3000).
- Em outro terminal, na pasta do projeto: npx http-server frontend -p 5500.
- Acesse http://localhost:5500/login.html e entre com carlos@gmail.com / 123456.
- Testes: npm test (automatizado) ou abra backend/teste.http no VS Code e execute as requisições.

9. Alterações realizadas no código recebido

Item	O que foi feito
Autenticação	O middleware apenas verificava se existia algum cabeçalho "authorization" (qualquer texto passava). Agora há login real, JWT com expiração de 90 min, rota /login e verificação da assinatura.
Senha e CPF	Eram gravados em texto puro. Senha agora usa bcrypt e CPF usa AES-256-CBC; as listagens mascaram o CPF.
Cliente	Possuía campo "endereco" (não pedido) e não tinha telefone, e-mail/CPF únicos nem validações. Ganhou telefone celular, validações e CRUD completo (listar, consultar, atualizar, excluir).
Console	Não tinha número de série, tipo de serviço nem vínculo com cliente. Ganhou todos os campos, número de série criptografado, chave estrangeira e consulta por termo (a função consultarConsole existia vazia).
Agendamento	Não era vinculado a cliente/console, tinha "tipoServico" (que pertence ao console) e não havia atualização nem listagem. Agora tem dataHora, vínculos, cadastro, atualização e listagem ordenada, com as regras RN05 a RN08.
Listar clientes	A função retornava erro 500 "não implementada". Foi implementada.
Banco de dados	A conexão apontava para o banco "a2". Agora usa db_pixelfix (via .env) e há script SQL com tabelas, chaves e dados de exemplo.
Frontend	Não havia login, logout nem controle de sessão, e o menu não identificava o cliente. Foram criados login, menu superior com nome do cliente e botão Sair, proteção de páginas e telas de listagem/edição para clientes, consoles e agendamentos.
Teste da API	O teste.http usava token fictício. Foi reescrito com login real e cenários positivos/negativos, e foi criada uma suíte automatizada (npm test).

Decisões de projeto

- CPF e nº de série usam criptografia determinística (IV derivado por HMAC do valor): o mesmo valor gera o mesmo texto cifrado, o que permite impor unicidade e consultar por nº de série sem guardar o dado em claro. A senha usa hash bcrypt, que é irreversível.
- A chave de criptografia e o segredo do JWT ficam no .env e devem ser trocados em produção.
- O horário do agendamento é tratado como horário de Brasília, sem conversão de fuso.